

Application Security



Input Validation Library 1.doc

Prepared by:

Lame_coyote

Lame_coyote009@users.sourceforge.net

Updated: 08/05/2005

Version: 0.1 RC 1

Company Name
Company Address
Location

Table of Contents

Table of Contents.....	2
1 Introduction.....	3
2 Scope, methodologies and limitations.....	3
2.1 Scope.....	3
2.2 Methodologies.....	3
2.3 Limitations.....	3
3 Application description.....	3
3.1 What does the library do?.....	3
3.2 Functionality.....	4
3.3 Technologies.....	5
4 Usage.....	5
4.1 Initial Setup and configuration.....	6
4.2 Usage of libinval.so in C/C++ projects.....	7
4.3 Usage of libinval.so in Perl projects.....	7
5 API.....	7
6 Internals.....	9
6.1 PCREBinGen.....	10
6.2 libinval.so.....	11
6.3 PCREBinGen.xml.....	12
6.4 libinval.pcre.....	12
7 TODO:.....	13
7.1 Profile and optimize code.....	13
7.2 More testing.....	13
7.3 Binary data encryption and checksum validation.....	13
8 Test results.....	13
9 Summary.....	14

1 Introduction

It seems widely accepted that most application vulnerabilities exist because of poor input validation. In fact, it is estimated that over 90 percent of them relate to lack or poor input validation. Therefore, an Input Validation Library was developed to make a programmers life easier.

This document will attempt to describe in detail what this library does, how it does it, what problems it solves and what its limitations are.

2 Scope, methodologies and limitations

2.1 Scope

The Input Validation Library provides development teams and programmers with input validation capabilities by checking existence of input, length of input, value range checking for standard data types, single character validation for strings and PCRE (Perl Compatible Regular Expressions) checks.

2.2 Methodologies

The library was designed to work with C/C++ and Perl programs, however its use could be extended to any other systems, since those technologies are widely supported. The library was implemented in C++ using Object Oriented Methodologies. Its interface was made compatible with C so that any C application can use it by simply including its header file and linking with the library. Additionally, its interface was made available to Perl by compiling a wrapper that can be invoked by Perl the same way Perl would invoke any standard module.

2.3 Limitations

The library has only been tested as a unique running instance per computer, therefore for the moment it can only be used on one application per server. More testing will be required in order to find out if it can run multiple instances.

When the library validates input and the validation does not pass the test, this probably indicates some kind of tampering or hacking attempt. By design, the library does not know “who” is doing that, who is trying to hack or at least who is sending bad data. Therefore, it is the application's responsibility to perhaps make a note of that or act upon that information. Perhaps, a desirable “feature” would be to allow an application to pass this “extra” information to the library, so that the library can handle this in a “standard” way. At the moment the library only saves this information in a stats file.

3 Application description

3.1 What does this library do?

The following are the steps the library takes during input validation:

NOTE: “item” refers to a type of validation, “string” refers to the input to be validated.

- Ensure we have a string and that it is not empty
- Compare string size with global minimum and maximum requirements
- Compare string size with item's minimum and maximum requirements
- If item is a standard data type ensure it conforms to proper type and range
- If item is a standard data type and has a value range ensure string is within proper range
- If item is a string ensure all non alpha-numeric characters are valid
- If item is a string ensure any repeating non alpha-numeric characters are valid
- If item has a regular expression validate the string using the regular expression

If at any point a validation step does not pass, all further tests are terminated for that string. A more detailed explanation is provided in section 6.2.

3.2 Functionality

The Input Validation Library uses a binary file as its configuration which is compiled with a utility that the Enterprise Security Team developed. Let's discuss this utility first since it's the first part of this library.

The utility is called PCREBinGen and it's an executable file developed in C++. When PCREBinGen is run, it looks for a file named PCREBinGen.xml unless another name is given at the command line. A number of other parameters are also accepted as optional and those deal with settings that control how the XML file is to be processed. When PCREBinGen.xml is read, it is parsed and the data is processed internally to produce an internal document; this document contains global variables that were found in the xml file. If some settings were missing it will use its internal defaults.

The xml file must also contain any number of entries named ValidateItem. A ValidateItem entry must contain settings that control how the input validation library is to operate on the data that is to be validated. The format of the xml file is provided in Section 6 – Internals.

When the parsing is done PCREBinGen has an object which contains all global variables and all the ValidateItem entries it found. This utility will then process each ValidateItem and will invoke PCRE Library which it uses to compile any PCRE string found. The data produced by the PCRE compile function is appended to the ValidateItem object contained in the InputValidationDocument. When the utility finishes processing all items it creates a file called libinval.pcre. At this point PCREBinGen's work concludes.

The name of the library is libinval.so which stands for, 'lib' the standard unix library prefix, 'in' for input, and 'val' for validation. libinval.pcre is used by the Input Validation Library as its configuration file. The main reason that the data is compiled separately by PCREBinGen is to gain performance. What this means is that the PCRE expression is compiled only once and the data is saved for future use. Other systems compile the data each time a check is performed, this of course takes time, and although the time it takes is negligible when you add it all up, it can be a considerable length of time.

As a side effect the library configuration file is kept in binary format this appears to make the system more secure. However, is worth mentioning that in this version, although the data in the file is binary, not all of it is unreadable. Therefore, anyone with a binary file viewer can figure out some interesting things about the file. The library could be improved in the next release by including a hashing and check-sum function which would make the file unreadable.

At this point we have two files that comprise our Input Validation Library, the library itself and `libinval.pcre`. Usage information is demonstrated in the next chapter and configuration data is provided in the following chapters.

Once we have an application to use the library. Validation can be done for the following data types: (signed/unsigned) char, short int, int, long, long long; float, double, bool and of course zero terminated strings. Note: the library validates strings only, this means that standard data type validation is done first by converting the string. As a side effect from validating standard data types the library can also provide the converted data. However, when a string is validated for compliance with a standard data type the validated value can be returned as *the* standard data type value.

The library keeps counters on all validations performed, total, passed, failed, errors and other counters that are more specific. This information is saved to a file upon library exit. Developers that use the library are encouraged to save the counters to a file periodically, this is so that in the event of an application or system failure we have statistical information that will prove useful later.

3.3 Technologies

Gnu's g++ was used to compile the utility and library.

The PCRE Library from www.pcre.org was used to perform regular expression checks. This library is widely used by a large number of companies including HMS, google, etc... `libinval.so` is linked statically to PCRE library, therefore it is unnecessary to keep a local copy on the machine where the library is to be used. This decreases dependencies and facilitates the installation and use of the library. In other words developers do not need to install pcre, `libinval` has already been linked statically to it. Of course a disadvantage is that when pcre provides upgrades they will not be used by the library. However, the library can be provided with dynamic linkage, in that case it will benefit from pcre upgrades. If the library is linked dynamically and the pcre compiled data binary format should change, it may be necessary to reproduce the `libinval.pcre` file to adopt the new format.

PCRE is quite ubiquitous and offers very high performance. The recommended way of dealing with input as you may already know is *not* by filtering “invalid” characters, but by discarding everything that is not considered “valid.” This presents some problems if we were to compose regular expressions in this manner. The regular expressions would become necessary large or at least more complex, therefore affecting performance. We have addressed this issue by coding another mechanism that deals with discarding invalid characters.

In the xml configuration file, the security or sysadmin will have to type all “valid” characters in the globals section. This section will then be applied to all items in the xml configuration that do not have a “valid” characters section of its own. Additionally, there is another section that deals with “valid” repeatable characters. These are characters that can appear in a string contiguously. By design, all alpha-numeric characters (a-z 0-9) space and new line characters can be repeated. In the case of space and new line characters that is true only if you allow space and multiline characters in the configuration file, for that item. Therefore, if you expect any other repeatable characters to appear in a string, you must make sure they are added to the configuration file.

There are a few other things that are adjustable through the configuration file. Such as character case sensitivity. If you set an item to be case insensitive, its PCRE data will be compiled with that option, that should also speed things up.

The Apache project XERCES XML parser library was used for PCREBinGen only, the library does not deal with XML at all in order to keep a small footprint.

A logging system similar to the one used in the neti project from HMS was used for output of information and other messages from the library.

4 Usage

The system was designed to be simple to use. Although the library is relatively easy to use, what takes the most time is editing the PCREBinGen.xml configuration file used to produce the binary file. This must be done very carefully and tested thoroughly to insure proper input validation. Fortunately, the Enterprise Security Team would produce prepackaged ValidateItems that would be useful in most situations and applications, and only those times deemed necessary should we produce new entries.

The xml validation file should *not* contain a comprehensive list of all the ValidateItem entries. Each application should have its own configuration file. This would make the library more efficient in its usage of memory, so that only the items that are necessary to validate on a per application basis would be kept in RAM.

4.1 Initial Setup and configuration

Before anyone attempts to use the library, the teams must agree on what's going to be validated and craft the necessary regular expressions carefully and test them thoroughly. Failure to do so may result in either a possible vulnerability or denial of service. After this step is completed, the Enterprise Security Team will prepare a PCREBinGen.xml file with the necessary information, then a libinval.pcre will be generated and sent to the development team.

Initially there should be 3 files that are necessary;

- libinval.so: This is the library file and it must be found in either of these locations; /usr/lib or /usr/local/lib. Alternatively, the file can be copied anywhere and a symbolic link pointed to it can be placed in any of the above mentioned locations.

- `libinval.pcre`: This is the configuration file and it must be found in either of these two locations; `/etc/libinval/libinval.pcre` or `/usr/local/etc/libinval.pcre`
- The other file is `libinval.h` header file for C/C++ projects or `InVal.pm` Perl Module for Perl projects. Actually, in Perl there are other files that must be copied in order to use `InVal.pm` module which is a wrapper. Alternatively, Perl programmers can use inline C embedded in perl code, which may improve performance and does not necessitate the extra Perl files.

At this point, the development team must know what is contained in the binary file, that is, the global defaults and each of the `ValidateItem` entries. Each of this entry has a name and that is how the library identifies each entry which is a type of validation.

In order to increase the library's performance, a different way of addressing the entries was devised, instead of using its name. Each entry is addressed by an ID. This ID is not known until runtime, (well, it is really the numerical order of each of the entries in the XML file, but this may change in future versions and must not be relied upon,) therefore, the programmer must obtain each entry ID based on its name. The names are case sensitive and can be things like Email, Name, Age, PhoneNumber, etc... The programmer must keep a reference to this ID throughout the runtime of their application. These IDs must be obtained each time the application is started.

Although, this may seem like an extra step, it is a necessary one in order to avoid calculating this internally each time by string comparison against possibly all entries. This, of course, was done in order to increase run time performance.

PERFORMANCE TIP: Put items that are to be validated the most in the beginning of the xml file. This will save a few cpu ticks.

The input validation library outputs information to `STDOUT` and `STDERR` and by default will output at a logging level of `LOG_INFO` (please see next chapter for meaning.) Programmers may do as they please with this information, i.e. redirect to system log or capture output.

The library tries to write statistic information to a file `/var/log/libinval.stats`, make sure that the library or program has permissions to write to that location. Otherwise the stats will be saved in `/var/tmp/libinval.stats`

The stats file contains a header with enough information to understand the format. The content is text. The function `iv_save_stats()` must be called periodically, otherwise stats may be lost. Stats information is crucial in order for the teams to understand what's been going on during input validation and to determine if adjustments to the configuration file will be necessary or to act upon the information learned in other ways.

4.2 Usage of *libinval.so* in C/C++ projects

Project preparation:

Add: **`#include "libinval.h"`** to the project or file that would call library routines.

Program initialization:

Call: ***idItem = iv_get_validate_item_id("ValidateItem name");***
For each item. Let's keep in mind that at this point the programmer knows what types of validation exist and knows them by name. If returned value is -1 an error occurred or the item was not found.

Input Validation:

Call: ***result = iv_validate_item(szStringToValidate, idItem);***

Result:

1 = Validation Passed

0 = Validation Failed

-1 = An error condition was encountered

4.3 Usage of libinval.so in Perl projects

The syntax would be similar to the one above. Perl programmers can use inline C or can use InVal.pm which is a wrapper. When using the wrapper, there are other files which must be copied into either the Perl system library/module directory or a user local Perl directory.

The extra files are:

Inval.pm

in: /usr/lib/perl5/site_perl/5.8.6/i686-linux/

and:

InVal.bs autosplit.ix Inval.so

in: /usr/lib/perl5/site_perl/5.8.6/i686-linux/auto/InVal

Note: in our tests, when using the wrapper, there was a time increase of about 95% than when using C. We have not tested inline C in Perl yet but it is presumably faster.

5 API

From libinval.h

```
//return values
#define IV_VALIDATION_PASSED 1
#define IV_VALIDATION_FAILED 0
#define IV_VALIDATION_ERROR -1

//log options
enum
{
    LOG_FATAL, //0
    LOG_CRIT,  //1
    LOG_ERR,   //2 * default
    LOG_WARN,  //3
    LOG_NOTICE, //4
    LOG_INFO,  //5
    LOG_DEBUG  //6
};
```


Depending on the logging level selected, the library will output anything equal to or less than the level selected. Therefore, do not use LOG_DEBUG in situations other than testing, doing so will bring the library to a crawl, as it will have to display information on just about anything it's doing. Although, LOG_INFO is the default, it is recommended that the programmers set the log level to LOG_ERR. It was done this way in order to make the programmer aware of the logging level selected. In a “real life” situation the output may be superfluous since the library returns a value with the result of the operation and that is the only thing that should be interesting. This is for performance reasons as well. When the library is compiled for release, LOG_DEBUG mode is disabled, this allowed us to gain an 8% performance increase.

```
#####

//INPUT VALIDATION
int iv_get_validate_item_id (char *);
int iv_validate_item (char *val, int id);

int iv_save_stats(void);

//optional
void iv_set_log_level( int level );
iv_set_log_level() can be used to adjust the library's log output level.
```

```
char * iv_get_last_error (void);
```

Error conditions can happen and when they do, a programmer is free to call this function to obtain perhaps more detailed information on why the error occurred.

```
//debugging, these functions only do something when compiled in debug mode
void iv_dump (void);
void iv_dump_stats (void);
```

These functions will output the global defaults and each of the ValidateItem entries and library validation counters respectively.

6 Internals

There are two C++ classes which are common to PCREBinGen and libinval.so: InputValidatorDocument and InputValidatorValidateItem. Details on these and the other classes will be provided in subsection 6.5. The following is a list of the main classes and their respective explanation:

- **PCREBinGenHandlers**

Derived from xerces xml parser handler class. Handles parsing and has functions that are called by the xerces system to notify the application and provide data. It uses other classes from xerces and it has the class object InputValidatorXMLReader, which it uses as a temporary buffer to save data from the xml file PCREBinGen.xml or the one selected by the user.

- **InputValidatorXMLReader**

When the parser finishes reading the xml file this class will have the xml data in a InputValidatorDocument object. So, the only purpose of this class is to intercept the data the parser throws at us and to build the InputValidatorDocument object. This class also has a set of defaults that can and will be used should the xml file be found incomplete, to a point.

- **InputValidatorDocument**

This class is the most important one for both the utility and the library. It contains the globals and all ValidateItem entries including any invalid entries in class objects InputValidatorValidateItem in a stl vector. This class has the capability so serialize itself, that is, given a file name (hopefully, libinval.pcre,) it will load the file and build itself into existence complete with globals and ValidateItem entries. It can also save itself, (really, no pun intended) to another file. The class is versioned and it stores PCRE library version information in the binary file.

- **InputValidatorValidateItem**

These objects are managed by InputValidatorDocument and get their content from the xml file ValidateItem entries. InputValidatorValidateItems actually do the input validation. They has data type knowledge and since all data is private, it has public accessor methods. These objects keep binary data that is produced by the PCRE library's pcre_compile2 and pcre_study. The actual data compilation is orchestrated by PCRECompiler.

- **PCRECompiler**

This class iterates through all InputValidatorValidateItem found in InputValidatorDocument m_sPCRE class member data and calls the functions mentioned above. If the function call pcre_compile2 succeeds information on the compile data is obtained such as its length. The data and information is saved in the InputValidatorValidateItem. Then this object calls pcre_study which is a function that will return extra data that will cause the input validation to be faster in some circumstances. If the study function returns any additional data, information about it is obtained and the length and extra data are saved.

- **BinSerializer**

This class is used by PCREBinGen to save and load files.

- **StdDataTypes**

This class is used to handle standard data types validation and conversions. It is a wrapper for standard C functions.

- **ErrorHandler**

This class is used to obtain system and exception error information and to process it in a standard and modular way. It also serves as a buffer for errors, so that later they can be dealt with accordingly.

6.1 PCREBinGen

This utility started from one of the samples included in the Xerces samples directory to save a lot of time. It was built from SAXPrint. SAXPrint will take a xml file and other parameters from the command line, will parse the file and output to STDOUT. It supports several character encodings and other options. Note that all you need to do is run the utility and these are just options, by default it will use PCREBinGen.xml from the current directory.

Usage:

```

"\nUsage:\n"
"  PCREBinGen [options] [XMLFile]\n\n"
"This program reads in InputValidator structures and compiles them \n"
"into binary format and outputs to a file for later use with the Input\n"
"Validation Library\n\n"
"Options:\n"
"  XMLFile [ " <<
g_pProgramName <<
".xml * ]\n"
"  -u=xxx    Handle unrepresentable chars [fail | rep | ref*].\n"
"  -v=xxx    Validation scheme [always | never | auto*].\n"
"  -n        Enable namespace processing.\n"
"  -s        Enable schema processing.\n"
"  -f        Enable full schema constraint checking.\n"
"  -x=XXX    Use a particular encoding for output (LATIN1*).\n"
"  -?        Show this help.\n\n"
"  * = Default if not provided explicitly.\n\n"
"The parser has intrinsic support for the following encodings:\n"
"  UTF-8, USASCII, ISO8859-1, UTF-16[BL]E, UCS-4[BL]E,\n"
"  WINDOWS-1252, IBM1140, IBM037, IBM1047.\n"

```

When PCREBinGen is run it starts parsing the command line and sets up various parameters and class objects that it will later use. If the appropriate parameters are input and the xml file is found it is used to “populate” InputValidatorDocument with Globals and ValidateItem entries.

A pointer to InputValidatorXMLReader is obtained from the PCREBinGenHandler object after parsing completes and a PCRECompiler object is invoked with the pointer as a parameter.

PCRECompiler.Compile(); is called to produce the binary data that the libinval.so will utilize subsequently. When the compilation concludes the class objects and PCRE binary data are saved to libinval.pcre or another file name by BinSerializer.Serialize(), and if that's the case it should be renamed to libinval.pcre in order to be used by libinval.so

6.2 libinval.so

When the library starts, it instantiates an object based on InputValidatorDocument, which at that time is empty. It has global variables with hard coded values to find libinval.pcre in the locations mentioned in the configuration section and the default logging level.

When the library is initialized, it attempts to locate its configuration file and outputs information regarding this operation. *InputValidatorDocument.Init()*; is called which handles any initialization. Then *InputValidatorDocument.LoadDocument("/location/to/libinval.pcre")*; is called which will cause the class object to build itself based on the configuration file.

At this point the library is ready for use.

The programmer would then call:

```
id = iv_get_validate_item_id( "ValidateItemName" );
```

Which is just a wrapper for:

```
int InputValidatorDocument::GetItemID (char *szItemName);
```

When the programmer is done obtaining Ids, Input Validation proceeds as follows:

First, please see next subsection to understand the main document structure.

When the programmer calls: `iv_validate_item(char* val, int id);`

The following happens:

- At InputValidatorDocument:
 - Ensure item's ID is within proper range
 - Ensure we have a string and it is not empty
 - For loop to find string size until global maximum size, so as not to exceed the global maximum string size
 - Compare string size with global minimum and maximum size
- At InputValidatorValidateItem:
 - If this InputValidatorValidateItem item has errors exit, such as bad PCRE compiled data, this could only occur in exceptional circumstances, for example system runs out of memory or something like that.
 - Compare string size with item's minimum and maximum size
 - If item is a standard data type:
 - Ensure it conforms to proper type and range
 - If item has a value range ensure it's within proper range
 - If item is a string:
 - Ensure all characters are valid, while doing so, save repeated characters in a buffer, to speed things up a bit
 - Ensure any repeating characters are valid
 - If item allows spaces, well, allow spaces
 - If item allows multiline, well, allow multiline then
 - If item has a PCRE validate using the regular expression

6.3 PCREBinGen.xml

Document structure:

Tag and color meanings: R:red=required, O:blue=optional overrides global, The rest are optional for the ValidateItem entries and all are required for globals so they are not tagged. If an entry is not found a default is used from within the class.

Note: The tag names (within <>) must match and are *case sensitive*.

```
<InputValidator> Document Root
  <Globals> Globals container
    <MinStringSize>0</MinStringSize> Minimum string size
    <MaxStringSize>255</MaxStringSize> Maximum string size
    <Caseless>true</Caseless> PCRE check will be case sensitive
if true
    <HasEncodedChars>>false</HasEncodedChars> Unused
```

```

        <ValidExtraChars>@#$/ValidExtraChars> Unused
        <NumericChars>123/NumericChars> Unused
        <AlphaChars>abc/AlphaChars> Unused
        <ValidSequences/ > Unused
        <AllowedRepeatingChars>abc/AllowedRepeatingChars>
    </Globals>
    <ValidateItem> List of entries for a unique type of validation
R    <Name>Name</Name> Name identifier, used by iv_get_validate_item_id()
R    <Type>string</Type> Type identifier:(string,int,long,float...)[C
data types]
R    <PCRE>[A-Za-z]</PCRE> Perl Compatible Regular Expression
0    <MinSize>2</MinSize>
0    <MaxSize>50</MaxSize>
0    <Caseless>true</Caseless>
    <Multiline>>false</Multiline>
    <AllowSpaces>>false </AllowSpaces>
    <MinValue>0</MinValue> Applicable only to standard data types
    <MaxValue>0</MaxValue> Applicable only to standard data types
0    <AllowedRepeatingChars/>
0    <ValidExtraChars/>
0    <ValidSequences/> Unused
    </ValidateItem>
<InputValidator>

```

6.4 libinval.pcre

This file contains all the data mentioned above. For version 1 this file contains the data from the class object in the way the data is handled. For example, strings are readable. This normally would not pose a problem, since it is considered that security through obscurity is not the best way of implementing security. However, we feel that the library can be improved by adding a hashing mechanism, so that the data is made unreadable and it would also contain checksum information in the file.

7 TODO:

7.1 More testing

7.2 Binary data encryption and checksum validation

8 Test results

Computer: DELL Inspiron 6000 RAM 1GB, avg CPU Load was 95%

```

processor      : 0
vendor_id     : GenuineIntel
cpu family    : 6
model         : 13
model name    : Intel(R) Pentium(R) M processor 1.60GHz
stepping      : 8
cpu MHz       : 1596.370
cache size    : 2048 KB
physical id   : 0

```

```
siblings      : 1
fdiv_bug      : no
hlt_bug       : no
f00f_bug      : no
coma_bug      : no
fpu           : yes
fpu_exception : yes
cpuid level   : 2
wp            : yes
flags         : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat clflush
dts acpi mmx fxsr sse sse2 ss tm pbe nx est tm2
bogomips      : 3137.53
```

OS: Gentoo Linux kernel 2.6.11-hardened-r15 #1 SMP Wed Jul 6 11:13:31 UTC 2005
i686 Intel(R) Pentium(R) M processor 1.60GHz GenuineIntel GNU/Linux

The tests were performed with a test utility program written in C. This program has a number of arrays of 10 items each. There is one array for each of the standard data types and one array for each of the items in our test xml configuration file. A total of 20, 13 standard data types and 7 test items. Three for loops, the first one being the control loop which number you specify at the command line of the test program. The second loop, calls the 7 test items in a random fashion, a total of 10 times. The third loop runs also 10 times in sequential order and tests the 7 items and the 13 standard data types. So, each time the loop number one runs it performs 140 tests.

The data in the arrays is a combination of valid and invalid data, some strings being empty, some exceed the allowed size. Strings for standard data types have valid and invalid ranges as well as “bad” characters. Some strings are large. The largest string tested exceeded 65,000 characters.

In any event. The release version showed that libinval.so is able to perform about 600,000 checks per second with the test data and the test configuration file. In “real life” tests the results will probably vary, depending on what kind of data will be validated, the complexity of the regular expressions and the combination of allowed characters.

Note that not all tests actually make it to a PCRE check. This is because the PCRE library test which is the Regular Expression check is done as the last step. Prior to that the string is checked for availability, correctness of length, in some cases correctness of value and existence of PCRE compiled binary data in the ValidateItem.

9 Summary

This library can provide HMS developers with an easy to use interface to PCRE. The library serves as a wrapper for PCRE (www.pcre.org) written by Philip Hazel, which appears to have done an excellent job in developing it. It is used and many well known applications such as Exim MTA, Python, Apache, PHP, KDE, Postfix, Analog, and many others. Google uses it as well and they developed a C++ wrapper for it.

Libinval.so is more than a wrapper because of the extra checks it performs. Those extra checks allow us to write much simpler regular expressions that will perform presumably much faster than doing it all through PCRE.

The use of this library facilitates programming, since the responsibility for input validation is delegated to the library for the most part. It serves as a way to organize and consolidate knowledge which can be shared company wide by virtue of using standardized validations, which are encapsulated in the binary configuration files.

The data that the library produces in the stats file can also prove quite useful. This data can be imported into databases for statistical analysis. This is useful because it would allow middle and upper management to make decisions based on the knowledge gained from statistical analysis. For example, suppose in March, April and June, you get a ratio of 5% input validation failures, the ratio is the comparison of failure with the number of HTTP requests, and, in July the ratio jumps to 10%, most likely, “something” is going on. And, of course, a number from that percentage is just harmless user error. But, what type of error? This can be learned in more detail by looking at the stats information. If, for example, you have lots of validation failures due to “invalid extra characters,” what’s going on here? At this point management directs systems and/or security departments to investigate and possibly obtain logs or other forensic information that could uncover the “identity” or at least the origin of the “bad” data. At that point it can be decided what can be done to remedy or mitigate this situation.

Another important factor to note is for example, an application is programmed to use the library and a vulnerability is later encountered. The vulnerability can be handled by editing the configuration and recompiling the binary file. Even in the extreme case, should the library have to be modified, it can be done so, without necessarily affecting the main application.

It is also necessary to say that this library will not be a cure for all, be everything for everybody, security all in one solution. It is the developers responsibility to code securely and the Enterprise Security Team to help doing so and to analyze code and applications for other types of vulnerabilities. The library was designed to validate input using PCRE and be as easy to use, inobstrusive and fast as possible.

We believe that it has met those requirements and our team highly recommends its use and inclusion into all applications company wide as soon as possible.